

JardimJá – Relatório de Integração Sprint 2 Lab. de DAMD

Ferramenta escolhida

Usei o RabbitMQ via CloudAMQP. A escolha foi prática: não precisa de instalação local, tem plano gratuito e o enunciado já cita como opção. A biblioteca [amqplib](#) conecta direto pelo Node.js

Organização do código

Três arquivos em [src/events](#):

- [connection.js](#) - abre e reutiliza a conexão com o RabbitMQ
- [publisher.js](#) - publica os eventos nas filas
- [consumer.js](#) - escuta as filas e processa as mensagens

Eventos implementados

Foram implementados três eventos, disparados nas rotas REST existentes:

- [nova_solicitacao](#) - publicado quando o cliente cria um pedido via [POST /solicitacoes](#)
- [solicitacao_aceita](#) - publicado quando o jardineiro aceita via [PATCH /solicitacoes/:id/status](#)
- [servico_concluido](#) - publicado quando o serviço é marcado como concluído

Overview					Messages			Message rates				+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack		
abtmqkoh	nova_solicitacao	classic	D Args default abtmqkoh-max-length	running	0	0	0	0.00/s	0.00/s	0.00/s		
abtmqkoh	servico_concluido	classic	D Args default abtmqkoh-max-length	running	0	0	0	0.00/s	0.00/s	0.00/s		
abtmqkoh	solicitacao_aceita	classic	D Args default abtmqkoh-max-length	running	0	0	0	0.00/s	0.00/s	0.00/s		

```
PS C:\PUCMINAS_Files\Git\LAB\Projeto-LAMD\jardimja-backend> node .\teste-rabbit.js
Conectado ao RabbitMQ
[Consumer] Escutando fila: nova_solicitacao
[Consumer] Escutando fila: solicitacao_aceita
[Consumer] Escutando fila: servico_concluido
[Publisher] Evento publicado em "nova_solicitacao": {
  solicitacaoId: 1,
  tipo: 'Corte de grama',
  clienteNome: 'Joao Teste',
  endereco: 'Rua das Flores, 100'
}
[Consumer] Mensagem recebida em "nova_solicitacao": {
  evento: 'nova_solicitacao',
  timestamp: '2020-05-24T00:58:21.846Z',
  dados: {
    solicitacaoId: 1,
    tipo: 'Corte de grama',
    clienteNome: 'Joao Teste',
    endereco: 'Rua das Flores, 100'
  }
}
[Publisher] Evento publicado em "solicitacao_aceita": { solicitacaoId: 1, jardineiroNome: 'Carlos Souza' }
[Consumer] Mensagem recebida em "solicitacao_aceita": {
  evento: 'solicitacao_aceita',
  timestamp: '2020-05-24T00:58:21.917Z',
  dados: { solicitacaoId: 1, jardineiroNome: 'Carlos Souza' }
}
[Consumer] Mensagem recebida em "servico_concluido": {
  evento: 'servico_concluido',
  timestamp: '2020-05-24T00:58:22.024Z',
  dados: { solicitacaoId: 1 }
}
```

Dificuldades

A conexão falhava porque eu usava [amqp://](#) em vez de [amqps://](#). Depois de corrigir no [.env](#) funcionou. Também precisei criar um singleton em [connection.js](#) para evitar abrir uma nova conexão a cada evento.

Conclusão

O backend publica os eventos e o consumer recebe as mensagens sem chamada REST entre eles. Nas próximas sprints os apps Flutter vão consumir esses eventos para atualizar a interface em tempo real.